

P-10: Inode Structures, Links and Memory-Mapped I/O

1. AIM

To investigate inode structures using Linux commands such as `ls -i`, `stat`, and `find`, create and analyze hard links and symbolic links, and study their impact on inode allocation. Also, to implement file reading and writing using `mmap()` and compare memory-mapped I/O with traditional `read()/write()` operations.

2. REQUIREMENTS

- Ubuntu/Linux operating system
 - GCC compiler
 - Linux Terminal
 - C programming language
 - Linux file-system commands
 - `mmap()` system call
-

PART A — INODE STRUCTURES

3. THEORY

An **inode** is a data structure maintained by the Linux file system that stores information about a file.

An inode contains information such as:

- File type
- File permissions
- File size
- Owner and group
- Timestamps
- Number of hard links

- Locations of file data blocks

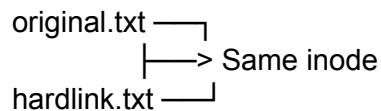
The inode **does not normally store the filename**. The filename is stored in a directory entry that points to the inode.

Important commands

Command	Purpose
<code>ls -li</code>	Displays the inode number
<code>stat</code>	Displays detailed file/inode information
<code>find</code>	Searches for files and directories

4. HARD LINK

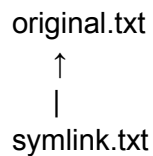
A **hard link** is another directory entry that points to the **same inode** as the original file.



Therefore, both files have the **same inode number**.

5. SYMBOLIC LINK

A **symbolic link (soft link)** is a separate file that contains a reference/path to another file.



The symbolic link has its **own inode**.

6. PROCEDURE

Step 1: Create a file

```
echo "This is inode practical." > original.txt
```

Step 2: Display inode number

```
ls -li original.txt
```

Step 3: Display detailed information

```
stat original.txt
```

Step 4: Create a hard link

```
ln original.txt hardlink.txt
```

Step 5: Check inode numbers

```
ls -li original.txt hardlink.txt
```

Both should show the **same inode number**.

Step 6: Create a symbolic link

```
ln -s original.txt symlink.txt
```

Step 7: Check inode numbers

```
ls -li original.txt hardlink.txt symlink.txt
```

The symbolic link will have a **different inode number** from the original file.

Step 8: Display detailed information

```
stat original.txt
stat hardlink.txt
stat symlink.txt
```

Step 9: Use **find** to locate the files

```
find . -inum $(ls -li original.txt | awk '{print $1}')
```

This finds directory entries having the same inode as **original.txt**.

7. OBSERVATION

- **original.txt** and **hardlink.txt** have the **same inode number**.
- **symlink.txt** has a **different inode number**.

- Creating a hard link increases the link count of the inode.
 - A symbolic link creates a separate inode containing a reference to the target.
 - Removing a hard link does not delete the actual file data as long as another hard link still exists.
-

PART B — MEMORY-MAPPED I/O USING `mmap()`

8. AIM

To implement file reading and writing using the `mmap()` system call and compare memory-mapped I/O with traditional `read()/write()` operations in terms of efficiency and implementation complexity.

9. THEORY

`mmap()` maps a file into the process's virtual memory.

Instead of repeatedly calling:

`read()` → buffer
`write()` → file

the program can access the mapped file directly through memory.

Traditional I/O

File
↓
`read()`
↓
Buffer
↓
`write()`
↓
File

Memory-mapped I/O

File



mmap()



Memory



Direct access

10. PROGRAM USING `mmap( )`

Create a C program:

nano mmap_copy.c

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat st;
    char *source;
    char *destination;
    int out;

    fd = open("source.txt", O_RDWR);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) < 0)
    {
        perror("fstat");
    }
}
```

```

    close(fd);
    return 1;
}

source = mmap(NULL, st.st_size,
               PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);

if (source == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

out = open("mmap_output.txt",
           O_RDWR | O_CREAT | O_TRUNC,
           0644);

if (out < 0)
{
    perror("open output");
    munmap(source, st.st_size);
    close(fd);
    return 1;
}

if (ftruncate(out, st.st_size) < 0)
{
    perror("ftruncate");
    close(out);
    munmap(source, st.st_size);
    close(fd);
    return 1;
}

destination = mmap(NULL, st.st_size,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED, out, 0);

if (destination == MAP_FAILED)
{
    perror("mmap output");
    close(out);
}

```

```
    munmap(source, st.st_size);
    close(fd);
    return 1;
}

memcpy(destination, source, st.st_size);

printf("File copied successfully using mmap().\n");

munmap(source, st.st_size);
munmap(destination, st.st_size);

close(fd);
close(out);

return 0;
}
```

11. EXECUTION

Compile:

```
gcc mmap_copy.c -o mmap_copy
```

Run:

```
./mmap_copy
```

Output

File copied successfully using mmap().

Check the copied file:

```
cat mmap_output.txt
```

The contents should be the same as `source.txt`.

12. COMPARISON

Traditional <code>read()/write()</code>	Memory-Mapped <code>mmap()</code>
Uses <code>read()</code> and <code>write()</code>	Uses <code>mmap()</code>
Data explicitly copied to/from buffers	File is mapped into memory
Simple for sequential I/O	Convenient for direct/random access
Requires explicit I/O operations	Allows memory-like access
Easier to understand initially	More complex setup
Suitable for many normal file operations	Useful for large files and repeated/random access

Efficiency

Memory-mapped I/O can reduce explicit copying between user buffers and system interfaces and can be efficient for suitable workloads. Actual performance depends on file size, access pattern, memory pressure, and the operating system's caching behavior.

13. RESULT

The inode structure of files was successfully investigated using `ls -li`, `stat`, and `find`. Hard and symbolic links were created and their inode numbers and link counts were examined.

A file-copy program using `mmap()` was successfully implemented. Memory-mapped I/O was compared with traditional `read()/write()` operations in terms of implementation and efficiency.

What to write in your record

AIM

To investigate inode structures using Linux commands, create and examine hard and symbolic links, and implement file reading and writing using `mmap()`. The performance of memory-mapped I/O is compared with traditional `read()/write()` operations.

RESULT

The inode structure of files was successfully examined using `ls -i`, `stat`, and `find`. A hard link and symbolic link were successfully created and verified. The `mmap()` program successfully copied the contents of a file, and the traditional `read()/write()` program also performed the copy successfully.

Performance Comparison

- `mmap()` real time: **0.003 s**
- `read()/write()` real time: **0.003 s**
- For the small test file, both methods showed approximately the same execution time.
- `mmap()` provides memory-mapped access to file contents and can be useful for larger files or repeated access.
- Traditional `read()/write()` uses explicit system calls to transfer data through a buffer.

Final Result

Thus, inode structures, hard links, symbolic links, and memory-mapped file I/O were successfully implemented and studied. ✓

```
This is an example of input redirection.
dup2() redirects standard input and output.
kruthi@LAPTOP-LP9QU3RR:~/p9$ nano redirect.c
kruthi@LAPTOP-LP9QU3RR:~/p9$ gcc redirect.c -o redirect
kruthi@LAPTOP-LP9QU3RR:~/p9$ ls
copy1.txt  destination.txt  input.txt      lowlevel_copy.c  redirect.c  standard_copy
copy2.txt  destination2.txt lowlevel_copy  redirect          source.txt  standard_copy.c
kruthi@LAPTOP-LP9QU3RR:~/p9$ ./redirect
kruthi@LAPTOP-LP9QU3RR:~/p9$ cat output.txt
This is an example of input redirection.
dup2() redirects standard input and output.
kruthi@LAPTOP-LP9QU3RR:~/p9$ cat < input.txt > shell_output.txt
kruthi@LAPTOP-LP9QU3RR:~/p9$ cat shell_output.txt
This is an example of input redirection.
dup2() redirects standard input and output.
kruthi@LAPTOP-LP9QU3RR:~/p9$ cd ..
kruthi@LAPTOP-LP9QU3RR:~$ mkdir p10
kruthi@LAPTOP-LP9QU3RR:~$ cd p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ pwd
/home/kruthi/p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This is inode practical." > original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt
2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ |
```

```

/home/kruthi/p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This is inode practical." > original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -li original.txt
2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln original.txt hardlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -li original.txt hardlink.txt
2218 hardlink.txt 2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ |

```

```

kruthi@LAPTOP-LP9QU3RR:~/p9$ cat shell_output.txt
This is an example of input redirection.
dup2() redirects standard input and output.
kruthi@LAPTOP-LP9QU3RR:~/p9$ cd ..
kruthi@LAPTOP-LP9QU3RR:~$ mkdir p10
kruthi@LAPTOP-LP9QU3RR:~$ cd p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ pwd
/home/kruthi/p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This is inode practical." > original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -li original.txt
2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln original.txt hardlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -li original.txt hardlink.txt
2218 hardlink.txt 2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln -s original.txt symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -li original.txt hardlink.txt symlink.txt
2218 hardlink.txt 2218 original.txt 2219 symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -l original.txt hardlink.txt symlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 hardlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 original.txt
lrwxrwxrwx 1 kruthi kruthi 12 Sep 28 08:36 symlink.txt -> original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ |

```

```
kruthi@LAPTOP-LP9QU3RR:~$ mkdir p10
kruthi@LAPTOP-LP9QU3RR:~$ cd p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ pwd
/home/kruthi/p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This is inode practical." > original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt
2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln original.txt hardlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt hardlink.txt
2218 hardlink.txt 2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln -s original.txt symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt hardlink.txt symlink.txt
2218 hardlink.txt 2218 original.txt 2219 symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -l original.txt hardlink.txt symlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 hardlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 original.txt
lrwxrwxrwx 1 kruthi kruthi 12 Sep 28 08:36 symlink.txt -> original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ find . -inum 2218
./hardlink.txt
./original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ find . -inum 2219
./symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat hardlink.txt
This is inode practical.
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat symlink.txt
This is inode practical.
kruthi@LAPTOP-LP9QU3RR:~/p10$ |
```

```

/home/kruthi/p10
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This is inode practical." > original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt
2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln original.txt hardlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt hardlink.txt
2218 hardlink.txt 2218 original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ln -s original.txt symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -i original.txt hardlink.txt symlink.txt
2218 hardlink.txt 2218 original.txt 2219 symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls -l original.txt hardlink.txt symlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 hardlink.txt
-rw-r--r-- 2 kruthi kruthi 25 Sep 28 08:35 original.txt
lrwxrwxrwx 1 kruthi kruthi 12 Sep 28 08:36 symlink.txt -> original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ find . -inum 2218
./hardlink.txt
./original.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ find . -inum 2219
./symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat hardlink.txt
This is inode practical.
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat symlink.txt
This is inode practical.
kruthi@LAPTOP-LP9QU3RR:~/p10$ echo "This file is used to demonstrate memory mapped I/O using mmap()."
" > source.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat source.txt
This file is used to demonstrate memory mapped I/O using mmap().
kruthi@LAPTOP-LP9QU3RR:~/p10$ nano mmap_copy.c
kruthi@LAPTOP-LP9QU3RR:~/p10$ gcc mmap_copy.c -o mmap_copy
kruthi@LAPTOP-LP9QU3RR:~/p10$ ls
hardlink.txt mmap_copy mmap_copy.c original.txt source.txt symlink.txt
kruthi@LAPTOP-LP9QU3RR:~/p10$ ./mmap_copy
File copied successfully using mmap().
kruthi@LAPTOP-LP9QU3RR:~/p10$ cat mmap_output.txt
This file is used to demonstrate memory mapped I/O using mmap().
kruthi@LAPTOP-LP9QU3RR:~/p10$ |

```